



INSTITUTO SUDAMERICANO

Carrera: Desarrollo de Software

Materia: Programación orientada a objetos

Tema: Backend (piensa)

Ciclo: 2do

Estudiantes: Pablo Ortega, Andrés Gutiérrez

Docente: Carolina Aldas

Fecha de entrega: 01 de diciembre de 2025

Ciudad: Cuenca

1 CREAR AUTH (LOGIN AND REGISTER)

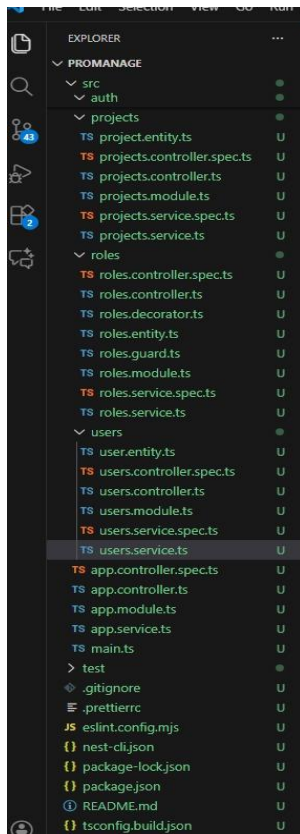
MIN 2 ROLES

PRUEBAS EN POSTMAN

ENVIAR DOCUMENTACION: CAPTURAS, EXPLICACIÓN, Y LINK DEL REPOSITORIO

1 Estructura de mi proyecto backend

Empezamos trabajando desde nuestra estructura que ya teníamos creada



2 Configuración de nuestra Base de Datos (PostgreSQL + TypeORM)

```

roles.service.ts | TS public.decorator.ts | TS jwt-auth.guard.ts | TS auth.controller.ts | TS app.module.ts | TS roles.module.ts
c > TS app.module.ts > % AppModule
1 import { Module } from '@nestjs/common';
2 import { TypeOrmModule } from '@nestjs/typeorm';
3 import { APP_GUARD } from '@nestjs/core';
4
5 import { AppController } from './app.controller';
6 import { AppService } from './app.service';
7
8 import { User } from './users/user.entity';
9 import { Role } from './roles/roles.entity';
10 import { Project } from './projects/project.entity';
11
12 import { AuthModule } from './auth/auth.module';
13 import { UsersModule } from './users/users.module';
14 import { RolesModule } from './roles/roles.module';
15 import { ProjectsModule } from './projects/projects.module';
16
17 import { RolesGuard } from './roles/roles.guard';
18 import { JwtAuthGuard } from './auth/jwt-auth.guard';
19
20 @Module({
21   imports: [
22     TypeOrmModule.forRoot({
23       type: 'postgres',
24       host: 'localhost',
25       port: 5432,
26       username: 'postgres',
27       password: '123456',
28       database: 'promanage',
29       entities: [User, Role, Project],
30       synchronize: false,
31     }),
32
33     AuthModule,
34     UsersModule,
35     RolesModule,
36     ProjectsModule,
37   ],
38   controllers: [AppController],
39   providers: [
40     AppService,
41
42     {
43       provide: APP_GUARD,
44       useClass: JwtAuthGuard,
45     },
46   ],
47
48

```

¿Para qué sirve?

- Conecta NestJS con PostgreSQL
- Permite mapear tablas como entidades
- TypeORM maneja las relaciones.

3 Entidades de nuestro Proyecto

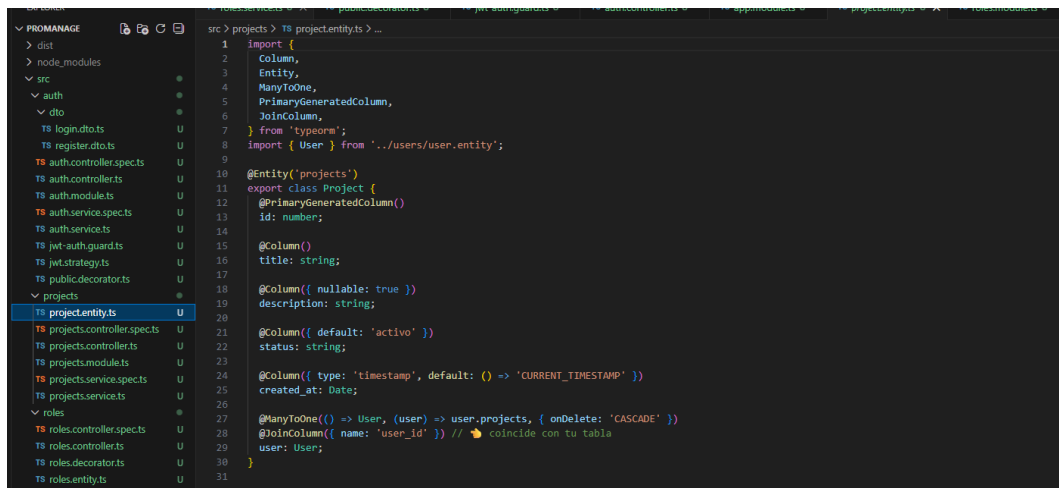
User.entity.ts

```
1 import {
2   Column,
3   Entity,
4   ManyToOne,
5   OneToMany,
6   PrimaryGeneratedColumn,
7   JoinColumn,
8 } from 'typeorm';
9 import { Role } from '../roles/roles.entity';
10 import { Project } from '../projects/project.entity';
11
12 @Entity('users')
13 export class User {
14   @PrimaryGeneratedColumn()
15   id: number;
16
17   @Column()
18   name: string;
19
20   @Column({ unique: true })
21   email: string;
22
23   @Column()
24   password: string; // hash
25
26   @ManyToOne(() => Role, (role) => role.users, { nullable: true, eager: true })
27   @JoinColumn({ name: 'role_id' })
28   role: Role;
29
30   @OneToMany(() => Project, (project) => project.user)
31   projects: Project[];
32 }
```

Role.entity.ts

```
1 import {
2   Column,
3   Entity,
4   OneToMany,
5   PrimaryGeneratedColumn,
6 } from 'typeorm';
7 import { User } from '../users/user.entity';
8
9 @Entity('roles')
10 export class Role {
11   @PrimaryGeneratedColumn()
12   id: number;
13
14   @Column({ unique: true })
15   name: string; // 'admin' | 'colaborador' | 'usuario'
16
17   @OneToMany(() => User, (user) => user.role)
18   users: User[];
19 }
```

Project.entity.ts

A screenshot of a code editor showing a file explorer on the left with a tree view of project files. The file 'project.entity.ts' is selected under the 'projects' directory. The main editor area displays the TypeScript code for this file, which includes imports for decorators like @Column, @Entity, @PrimaryGeneratedColumn, and @JoinColumn. It defines a 'Project' class with attributes like 'id', 'title', 'description', 'status', and 'created_at', and a many-to-one relationship with the 'User' entity.

```
src > projects > TS project.entity.ts > ...
1  import {
2    Column,
3    Entity,
4    ManyToOne,
5    PrimaryGeneratedColumn,
6    JoinColumn,
7  } from 'typeorm';
8  import { User } from '../users/user.entity';
9
10 @Entity('projects')
11 export class Project {
12   @PrimaryGeneratedColumn()
13   id: number;
14
15   @Column()
16   title: string;
17
18   @Column({ nullable: true })
19   description: string;
20
21   @Column({ default: 'activo' })
22   status: string;
23
24   @Column({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP' })
25   created_at: Date;
26
27   @ManyToOne(() => User, (user) => user.projects, { onDelete: 'CASCADE' })
28   @JoinColumn({ name: 'user_id' }) // coincide con tu tabla
29   user: User;
30 }
31
```

4 Modulos del proyecto

¿Para qué sirve?

Organiza el proyecto por funcionalidades y mejora el mantenimiento.

Auth modulo.ts

A screenshot of a code editor showing a file explorer on the left with a tree view of project files. The file 'auth.module.ts' is selected under the 'auth' directory. The main editor area displays the TypeScript code for this file, which includes imports for Module, JwtModule, PassportModule, AuthService, AuthController, UsersModule, and JwtStrategy. It defines an 'AuthModule' class using the @Module decorator, specifying imports, providers, controllers, and exports.

```
src > auth > TS auth.module.ts > ...
1  import { Module } from '@nestjs/common';
2  import { JwtModule } from '@nestjs/jwt';
3  import { PassportModule } from '@nestjs/passport';
4
5  import { AuthService } from './auth.service';
6  import { AuthController } from './auth.controller';
7  import { UsersModule } from '../users/users.module';
8  import { JwtStrategy } from '../jwt.strategy';
9
10 @Module({
11   imports: [
12     UsersModule,
13     PassportModule,
14     JwtModule.register({
15       secret: 'mi_secreto_jwt', // mismo secret que en jwt.strategy.ts
16       signOptions: { expiresIn: '8h' },
17     }),
18   ],
19   providers: [AuthService, JwtStrategy],
20   controllers: [AuthController],
21   exports: [AuthService, JwtModule],
22 })
23 export class AuthModule {}
24
```

Usersmodule

```
1 import { Module } from '@nestjs/common';
2 import { TypeOrmModule } from '@nestjs/typeorm';
3 import { UsersController } from './users.controller';
4 import { UsersService } from './users.service';
5 import { User } from './user.entity';
6 import { RolesModule } from '../roles/roles.module';
7
8 @Module({
9   imports: [TypeOrmModule.forFeature([User]), RolesModule],
10  controllers: [UsersController],
11  providers: [UsersService],
12  exports: [UsersService],
13 })
14 export class UsersModule {}
15
```

Rolemodule.ts

```
1 import { Module } from '@nestjs/common';
2 import { TypeOrmModule } from '@nestjs/typeorm';
3 import { Role } from './roles.entity';
4 import { RolesService } from './roles.service';
5 import { RolesController } from './roles.controller';
6
7 @Module({
8   imports: [TypeOrmModule.forFeature([Role])],
9   providers: [RolesService],
10  controllers: [RolesController],
11  exports: [RolesService],
12 })
13 export class RolesModule {}
14
```

Project.module.ts

```
ORER ... TS roles.service.ts U X TS public.decorator.ts U TS jwt-auth.guard.ts U TS auth.controller.ts U
MANAGE MANAGE MANAGE MANAGE
code_modules
c
auth
dto
TS login.dto.ts U
TS register.dto.ts U
auth.controller.spec.ts U
auth.controller.ts U
auth.module.ts U
auth.service.spec.ts U
auth.service.ts U
jwt-auth.guard.ts U
jwt.strategy.ts U
public.decorator.ts U
projects
project.entity.ts U
projects.controller.spec.ts U
projects.controller.ts U
projects.module.ts U
projects.service.spec.ts U
projects.service.ts U

src > projects > TS projects.module.ts > ...
1 import { Module } from '@nestjs/common';
2 import { TypeOrmModule } from '@nestjs/typeorm';
3 import { ProjectsService } from '../projects.service';
4 import { ProjectsController } from '../projects.controller';
5 import { Project } from '../project.entity';
6
7 @Module({
8   imports: [TypeOrmModule.forFeature([Project])],
9   controllers: [ProjectsController],
10  providers: [ProjectsService],
11  exports: [ProjectsService],
12 })
13 export class ProjectsModule {}
14
```

S servicios

Project.service.ts

```
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository } from 'typeorm';
import { Project } from '../project.entity';
import { User } from '../users/user.entity';

@Injectable()
export class ProjectsService {
  constructor(
    @InjectRepository(Project)
    private readonly projectsRepo: Repository<Project>,
  ) {}

  // Crear un proyecto (por si lo usas desde un controller de creación)
  async createForUser(
    user: User,
    title: string,
    description?: string,
  ): Promise<Project> {
    const project = this.projectsRepo.create({
      title,
      description,
      user,
    });
  }
}
```

```

    });

    return this.projectsRepo.save(project);
  }

  // ◇ Usado por @Get('all') -> SOLO admin
  async findAll(): Promise<Project[]> {
    return this.projectsRepo.find({
      relations: ['user'],
      order: { created_at: 'DESC' },
    });
  }

  // ◇ Usado por @Get() -> usuario / colaborador / admin (solo los suyos)
  async findByUser(userId: number): Promise<Project[]> {
    return this.projectsRepo.find({
      where: { user: { id: userId } },
      relations: ['user'],
      order: { created_at: 'DESC' },
    });
  }

  // ◇ Usado por @Get('resumen') -> admin y colaborador
  async getResumen() {
    const total = await this.projectsRepo.count();
    const activos = await this.projectsRepo.count({ where: { status: 'activo' } });
  });
    const completados = await this.projectsRepo.count({ where: { status:
'completado' } });
    const archivados = await this.projectsRepo.count({ where: { status:
'archivado' } });

    return {
      total,
      activos,
      completados,
      archivados,
    };
  }
}

```

Roles.service.ts


```
src > roles > TS roles.service.ts > ...
1  import { Injectable } from '@nestjs/common';
2  import { InjectRepository } from '@nestjs/typeorm';
3  import { Repository } from 'typeorm';
4  import { Role } from './roles.entity';
5
6  @Injectable()
7  export class RolesService {
8    constructor(
9      @InjectRepository(Role)
10     private readonly rolesRepo: Repository<Role>,
11    ) {}
12
13     async findByName(name: 'admin' | 'colaborador' | 'usuario') {
14       return this.rolesRepo.findOne({ where: { name } });
15     }
16
17     findAll() {
18       return this.rolesRepo.find();
19     }
20   }
21
```

Roles.service y roles.module

El RolesService permite:

- Buscar roles por nombre (admin, usuario)
- Obtener los roles registrados

¿Para qué sirve?

Asignar el rol correcto al usuario durante el registro.

Users.service.ts

```

src > users > TS users.service.ts > *UsersService
1  import { Injectable } from '@nestjs/common';
2  import { InjectRepository } from '@nestjs/typeorm';
3  import { Repository } from 'typeorm';
4  import { User } from '../user.entity';
5  import { RolesService } from '../roles/roles.service';
6
7  @Injectable()
8  export class UsersService {
9    constructor(
10     @InjectRepository(User)
11     private readonly usersRepo: Repository<User>,
12     private readonly rolesService: RolesService,
13   ) {}
14
15   async createUser(data: {
16     name: string;
17     email: string;
18     password: string;
19     role?: string;
20   }): Promise<User> {
21     const roleName = (data.role as 'admin' | 'colaborador' | 'usuario') ?? 'usuario';
22
23     const roleEntity = await this.rolesService.findByName(roleName);
24     if (!roleEntity) {
25       throw new Error("No existe el rol '${roleName}' en la base de datos");
26     }
27
28     const newUser = this.usersRepo.create({
29       name: data.name,
30       email: data.email,
31       password: data.password,
32       role: roleEntity,
33     });
34
35     return this.usersRepo.save(newUser);
36   }
37
38   async findByEmail(email: string): Promise<User | null> {
39     const user = await this.usersRepo.findOne({
40       where: { email },
41       relations: ['role'],
42     });
43     return user ?? null;
44   }
45 }

```

Users.controller

```

src > users > TS users.controller.ts > ...
1  import { Controller, Get, Req } from '@nestjs/common';
2  import { Roles } from '../roles/roles.decorator';
3  import express from 'express';
4
5  @Controller('users')
6  export class UsersController {
7    @Get()
8    @Roles('admin')
9    findAll() {
10     return { message: 'Solo el ADMIN puede ver esta ruta' };
11   }
12
13   @Get('me')
14   @Roles('admin', 'usuario')
15   getProfile(@Req() req: express.Request) {
16     return {
17       message: 'Perfil del usuario logueado',
18       user: req.user,
19     };
20   }
21 }
22

```

Users.service.ts y users.controller

El servicio del usuario permite:

- Crear usuario
- Buscar usuario por correo
- Manejar la relación con roles

¿Para qué sirve?

Es la lógica de negocio del usuario (registro automático con rol por defecto)

Crear decoradores

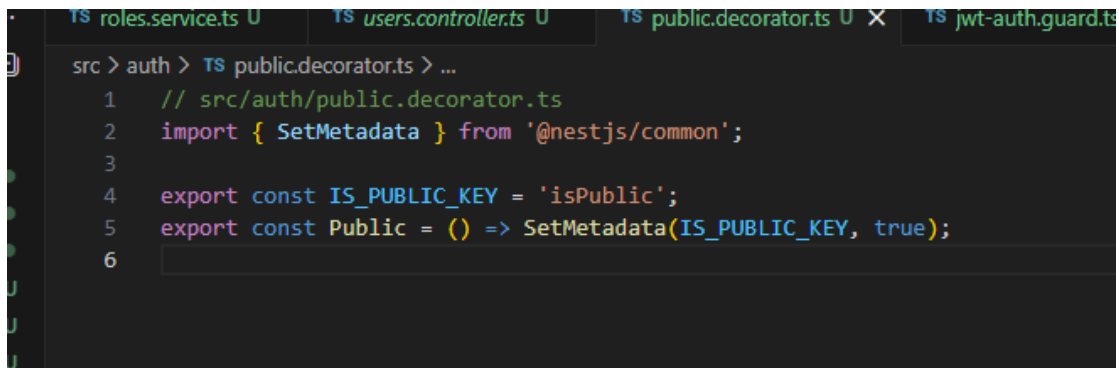
public.decorator.ts

Qué es:

Un decorador personalizado que se pone encima de ciertos endpoints como `@Public()`.

Para qué sirve:

- Marca una ruta como pública.
- Le dice al `JwtAuthGuard` que esa ruta NO necesita token.
- Se usa en `/auth/register` y `/auth/login` para que los usuarios puedan registrarse e iniciar sesión sin estar autenticados todavía.



```
src > auth > TS public.decorator.ts > ...
1 // src/auth/public.decorator.ts
2 import { SetMetadata } from '@nestjs/common';
3
4 export const IS_PUBLIC_KEY = 'isPublic';
5 export const Public = () => SetMetadata(IS_PUBLIC_KEY, true);
6
```

roles.decorator.ts

Qué es:

Un decorador personalizado para indicar qué roles pueden acceder a una ruta.

Para qué sirve:

- Permite escribir cosas como `@Roles('admin')` o `@Roles('admin', 'usuario')`.
- Guarda internamente la lista de roles requeridos para que el `RolesGuard` los lea.
- Es la forma “bonita y declarativa” de proteger rutas por rol.

```
TS roles.service.ts U TS public.decorator.ts U TS roles.decorator.ts U X TS jwt-auth.guard.ts U TS aut

src > roles > TS roles.decorator.ts > ...
1 import { SetMetadata } from '@nestjs/common';
2
3 export const ROLES_KEY = 'roles';
4 export const Roles = (...roles: string[]) => SetMetadata(ROLES_KEY, roles);
5
```

jwt-auth.guard.ts

Qué es:

Un guard (filtro de seguridad) que se ejecuta antes de entrar a cualquier ruta.

Para qué sirve:

- Revisa si la ruta es pública (`@Public()`).
 - Si es pública → deja pasar sin exigir token.
 - Si NO es pública → exige un token JWT válido.
- Si el token es correcto, deja pasar y adjunta al usuario en `request.user`.
- Protege todo el backend de accesos sin autenticarse.

```
src > auth > TS jwt-auth.guard.ts > ...
2 import { ExecutionContext, Injectable } from '@nestjs/common';
3 import { Reflector } from '@nestjs/core';
4 import { AuthGuard } from '@nestjs/passport';
5 import { IS_PUBLIC_KEY } from '../public.decorator';
6
7 @Injectable()
8 export class JwtAuthGuard extends AuthGuard('jwt') {
9   constructor(private reflector: Reflector) {
10     super();
11   }
12
13   canActivate(context: ExecutionContext) {
14     // 🌟 Revisar si la ruta es pública
15     const isPublic = this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
16       context.getHandler(),
17       context.getClass(),
18     ]);
19
20     if (isPublic) {
21       // Si es pública, NO pedimos token
22       return true;
23     }
24
25     // Si no es pública, aplicamos el guard normal
26     return super.canActivate(context);
27   }
28 }
29
```

roles.guard.ts

Qué es:

Otro guard que se ejecuta después de que el usuario ya está autenticado (después del JWT).

Para qué sirve:

- Lee los roles requeridos de la ruta (mediante `@Roles()`).
- Compara el rol del usuario (`request.user.role`) con los roles permitidos.
- Si el usuario no tiene un rol permitido → responde con 403 No tienes permisos suficientes.
- Es el que hace la diferencia entre lo que puede hacer un **admin** y un **usuario**.

```
src > roles > ts roles.guard.ts > ...
1  import {
2    CanActivate,
3    ExecutionContext,
4    ForbiddenException,
5    Injectable,
6  } from '@nestjs/common';
7  import { Reflector } from '@nestjs/core';
8  import { ROLES_KEY } from './roles.decorator';
9
10 @Injectable()
11 export class RolesGuard implements CanActivate {
12   constructor(private reflector: Reflector) {}
13
14   canActivate(context: ExecutionContext): boolean {
15     const requiredRoles = this.reflector.getAllAndOverride<string[]>(
16       ROLES_KEY,
17       [context.getHandler(), context.getClass()],
18     );
19
20     if (!requiredRoles || requiredRoles.length === 0) {
21       return true;
22     }
23
24     const request = context.switchToHttp().getRequest();
25     const user = request.user;
26
27     if (!user || !user.role) {
28       throw new ForbiddenException('Usuario sin rol');
29     }
30
31     if (!requiredRoles.includes(user.role)) {
32       throw new ForbiddenException('No tienes permisos suficientes');
33     }
34
35     return true;
36   }
37 }
38
```

jwt.strategy.ts (esto También lo creamos)

Qué es:

La configuración que le dice a Nest cómo leer y validar el token JWT.

Para qué sirve:

- Indica de dónde sacar el token (del header `Authorization: Bearer token`).
- Valida el token usando el `secret`.
- A partir del payload del token, construye el objeto `user` que se guarda en `request.user`:
 - id
 - email
 - name
 - role
- Ese `user` es el que luego usan los guards y los controladores.

```

src > auth > TS jwt.strategy.ts > ...
1  import { Injectable } from '@nestjs/common';
2  import { PassportStrategy } from '@nestjs/passport';
3  import { ExtractJwt, Strategy } from 'passport-jwt';
4
5  @Injectable()
6  export class JwtStrategy extends PassportStrategy(Strategy) {
7    constructor() {
8      super({
9        jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
10       ignoreExpiration: false,
11       secretOrKey: 'mi_secreto_jwt',
12     });
13   }
14
15   async validate(payload: any) {
16     return {
17       id: payload.sub,
18       email: payload.email,
19       name: payload.name,
20       role: payload.role, // 'admin' | 'usuario'
21     };
22   }
23 }
24

```

register.dto.ts y login.dto.ts (esto también lo creamos)

Qué son:

Clases que representan la estructura de los datos que llegan en el body de las peticiones.

Para qué sirven:

- `register.dto.ts`: define qué datos se necesitan para registrar un usuario:
 - nombre
 - email
 - contraseña
 - opcionalmente rol
- `login.dto.ts`: define qué datos se necesitan para iniciar sesión:
 - email
 - contraseña
- Ordenan el código, ayudan a validar y documentar claramente qué espera cada endpoint.

register.dto.ts

```

src > auth > dto > TS register.dto.ts > ...
1   export class RegisterDto {
2       name: string;
3       email: string;
4       password: string;
5       role?: 'admin' | 'colaborador' | 'usuario';
6   }
7

```

login.dto.ts

```

src > auth > dto > TS login.dto.ts > ...
1   export class LoginDto {
2       email: string;
3       password: string;
4   }
5

```

auth.service.ts

Qué es:

La lógica principal de autenticación del sistema.

Para qué sirve:

- **register:**
 - Recibe los datos del DTO de registro.
 - Encripta la contraseña usando bcrypt.
 - Llama al `UserService` para crear el usuario en la base de datos con su rol.
- **validateUser:**
 - Busca el usuario por email.
 - Compara la contraseña en texto con el hash guardado usando bcrypt.
- **login:**
 - Genera el token JWT con los datos del usuario (id, email, nombre, rol).
 - Devuelve el `access_token` y los datos básicos del usuario.

Es literal el “cerebro” de la autenticación.

```

src > auth > ts auth.service.ts > login > payload
1 import { Injectable, ConflictException } from '@nestjsjs/common';
2 import { JwtService } from '@nestjsjs/jwt';
3 import { UsersService } from '../users/users.service';
4 import * as bcrypt from 'bcrypt';
5 import { User } from '../users/user.entity';
6 import { RegisterDto } from '../dto/register.dto';
7
8 @Injectable()
9 export class AuthService {
10   constructor(
11     private readonly usersService: UsersService,
12     private readonly jwtService: JwtService,
13   ) {}
14
15   async validateUser(email: string, pass: string): Promise<User | null> {
16     const user = await this.usersService.findByEmail(email);
17     if (!user) return null;
18
19     const isMatch = await bcrypt.compare(pass, user.password);
20     if (!isMatch) return null;
21
22     return user;
23   }
24
25   async login(user: User) {
26     const payload = {
27       sub: user.id,
28       email: user.email,
29       name: user.name,
30       role: user.role.name, // 'admin' | 'usuario'
31     };
32
33     return {
34       access_token: this.jwtService.sign(payload),
35       user: payload,
36     };
37   }
38
39   async register(dto: RegisterDto) {
40     const existing = await this.usersService.findByEmail(dto.email);
41     if (existing) {
42       throw new ConflictException('El correo ya está registrado');
43     }
44
45     const hashedPassword = await bcrypt.hash(dto.password, 10);
46
47     const newUser = await this.usersService.createUser({
48       name: dto.name,
49       email: dto.email,
50       password: hashedPassword,
51       role: 'usuario',
52     });
53   }
54 }

```

auth.controller.ts

Qué es:

El controlador que expone las rutas /auth/register y /auth/login.

Para qué sirve:

- POST /auth/register:
 - Ruta pública (@Public()).
 - Recibe los datos del usuario.
 - Llama a authService.register y devuelve un mensaje con el usuario registrado.
- POST /auth/login:
 - Ruta pública (@Public()).
 - Recibe email y contraseña.
 - Llama a authService.validateUser.
 - Si es válido, devuelve el token generado por authService.login.
 - Si no, devuelve error de credenciales inválidas.

Es la “puerta de entrada” para el registro y el login.


```

src > auth > TS auth.controllers > ...
1 // src/auth/auth.controller.ts
2 import { Body, Controller, Post, UnauthorizedException } from '@nestjs/common';
3 import { AuthService } from '../auth.service';
4 import { RegisterDto } from '../dto/register.dto';
5 import { LoginDto } from '../dto/login.dto';
6 import { Public } from '../public.decorator'; // 🐼 importa esto
7
8 @Controller('auth')
9 export class AuthController {
10   constructor(private readonly authService: AuthService) {}
11
12   // 🐼 Ruta pública: no necesita token
13   @Public()
14   @Post('register')
15   register(@Body() dto: RegisterDto) {
16     return this.authService.register(dto);
17   }
18
19   // 🐼 Ruta pública: no necesita token
20   @Public()
21   @Post('login')
22   async login(@Body() dto: LoginDto) {
23     const user = await this.authService.validateUser(dto.email, dto.password);
24     if (!user) {
25       throw new UnauthorizedException('Credenciales inválidas');
26     }
27     return this.authService.login(user);
28   }
29 }
30

```

users.service.ts

Qué es:

El servicio que maneja todo lo relacionado con la entidad User.

Para qué sirve (en autenticación y roles):

- Crear nuevos usuarios:
 - Asigna un rol por defecto (usuario) o el rol que venga en el DTO (admin, usuario, etc.).
 - Guarda la relación con la tabla roles.
- Buscar usuarios por email:
 - Es usado por auth.service para validar login.

Se encarga de hablar con la base de datos para todo lo relacionado a “users”.

```

src > users > TS users.services > UsersService
1  import { Injectable } from '@nestjs/common';
2  import { InjectRepository } from '@nestjs/typeorm';
3  import { Repository } from 'typeorm';
4  import { User } from '../user.entity';
5  import { RolesService } from '../roles/roles.service';
6
7  @Injectable()
8  export class UsersService {
9    constructor(
10     @InjectRepository(User)
11     private readonly usersRepo: Repository<User>,
12     private readonly rolesService: RolesService,
13   ) {}
14
15   async createUser(data: {
16     name: string;
17     email: string;
18     password: string;
19     role?: string;
20   }): Promise<User> {
21     const roleName = (data.role as 'admin' | 'colaborador' | 'usuario') ?? 'usuario';
22
23     const roleEntity = await this.rolesService.findByName(roleName);
24     if (!roleEntity) {
25       throw new Error("No existe el rol "${roleName}" en la base de datos");
26     }
27
28     const newUser = this.usersRepo.create({
29       name: data.name,
30       email: data.email,
31       password: data.password,
32       role: roleEntity,
33     });
34
35     return this.usersRepo.save(newUser);
36   }
37
38   async findByEmail(email: string): Promise<User | null> {
39     const user = await this.usersRepo.findOne({
40       where: { email },
41       relations: ['role'],
42     });
43     return user ?? null;
44   }
45 }
46

```

users.controller.ts

Qué es:

Controlador para rutas relacionadas con los usuarios.

Para qué sirve (en roles):

- Ruta /users:
 - Protegida con @Roles('admin').
 - Solo puede acceder un usuario con rol admin.
 - Se usó como prueba clara de restricción de rol.
- Ruta /users/me:
 - Protegida con @Roles('admin', 'usuario').
 - Permite tanto admin como usuario normal.
 - Devuelve el perfil del usuario logueado usando request.user.

Son los endpoints donde se ve realmente funcionando el sistema de roles

```

src > users > TS users.controller.ts > ...
1  import { Controller, Get, Req } from '@nestjs/common';
2  import { Roles } from '../roles/roles.decorator';
3  import express from 'express';
4
5  @Controller('users')
6  export class UsersController {
7      @Get()
8      @Roles('admin')
9      findAll() {
10         return { message: 'Solo el ADMIN puede ver esta ruta' };
11     }
12
13     @Get('me')
14     @Roles('admin', 'usuario')
15     getProfile(@Req() req: express.Request) {
16         return {
17             message: 'Perfil del usuario logueado',
18             user: req.user,
19         };
20     }
21 }
22

```

Seguridad

Uso de **bcrypt** (en autenticación)

Qué es:

Una librería para encriptar contraseñas.

Para qué sirve:

- En `register`:
 - Convierte la contraseña que escribe el usuario en un hash.
 - Guarda el hash en la BD, no la contraseña real.
- En `validateUser`:
 - Compara la contraseña que el usuario escribe en login con el hash guardado.

Aporta seguridad a las cuentas de los usuarios.

Uso de **JWT** con rol en el payload

Qué es:

El sistema de token que se usa para autenticación sin sesiones.

Para qué sirve:

- Después de login:
 - Crea un token con los datos del usuario.
 - Incluye explícitamente el `role` del usuario.
- En cada petición:

- El token se manda en el header.
- JwtStrategy lo decodifica y RolesGuard usa el role para decidir si puede entrar o no.

Gracias a esto puedes decir:

“Este token pertenece a un admin” o “Este token pertenece a un usuario normal”.

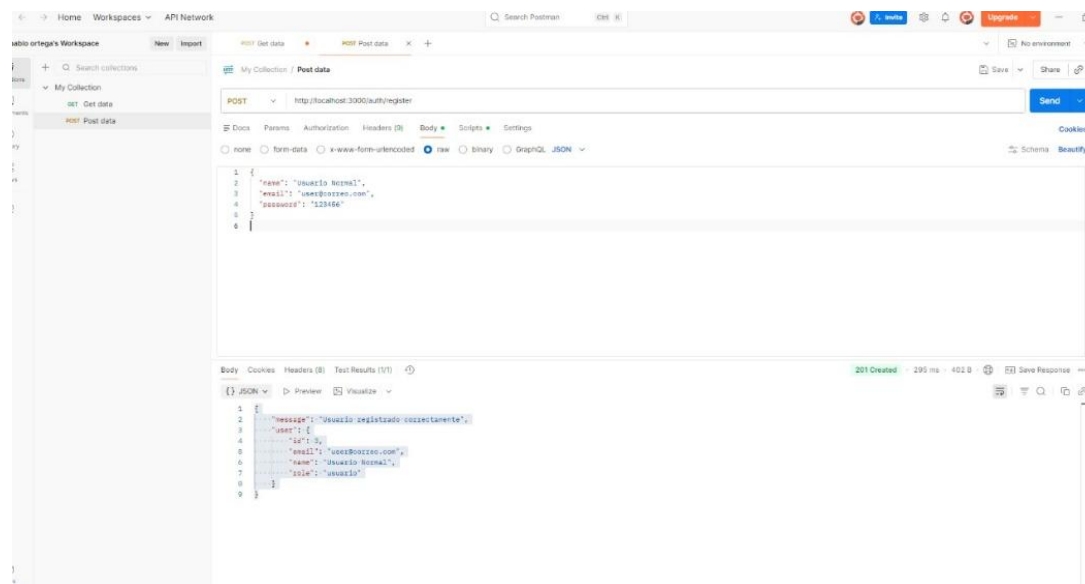
Pruebas en postman: Prueba de registrar un usuario

Se **registró bien** el usuario

Se guardó en la BD

Se le asignó correctamente el rol "**usuario**"

El método register y UsersService están funcionando



Prueba del login con usuario

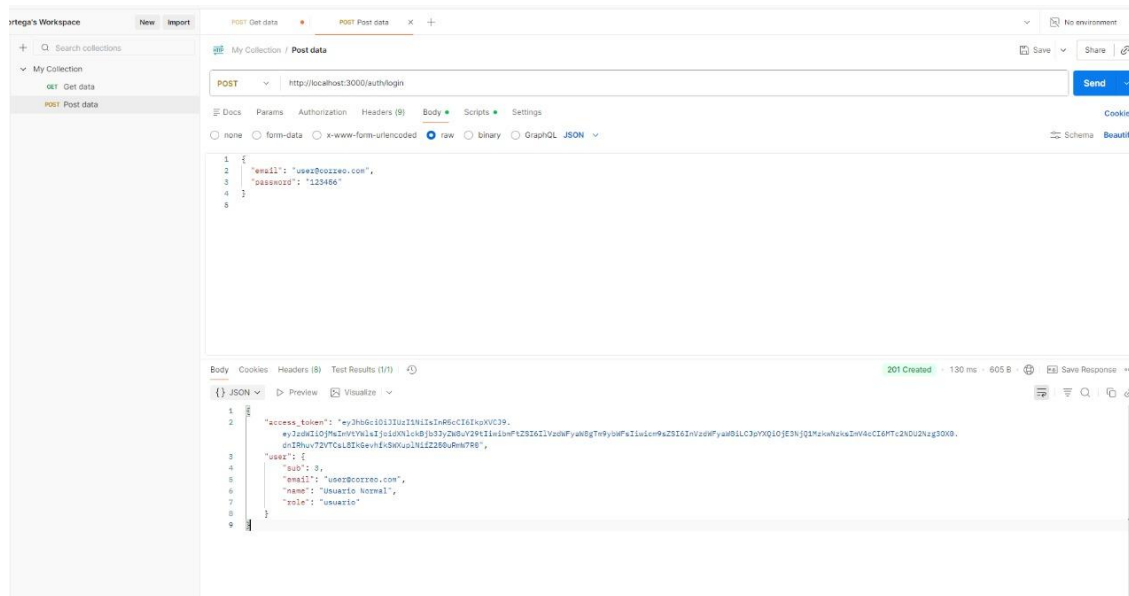
El login funciona

La validación bcrypt funciona

El usuario existe en la BD

El token JWT se generó correctamente

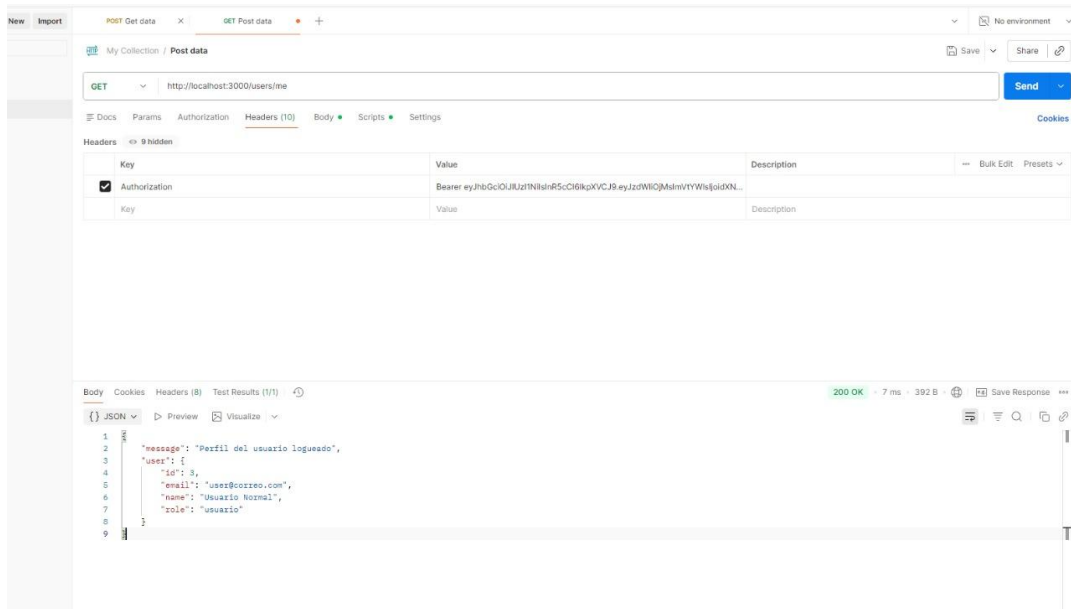
El rol se incluye dentro del payload



Probar users/me con el mismo token (usuario normal)

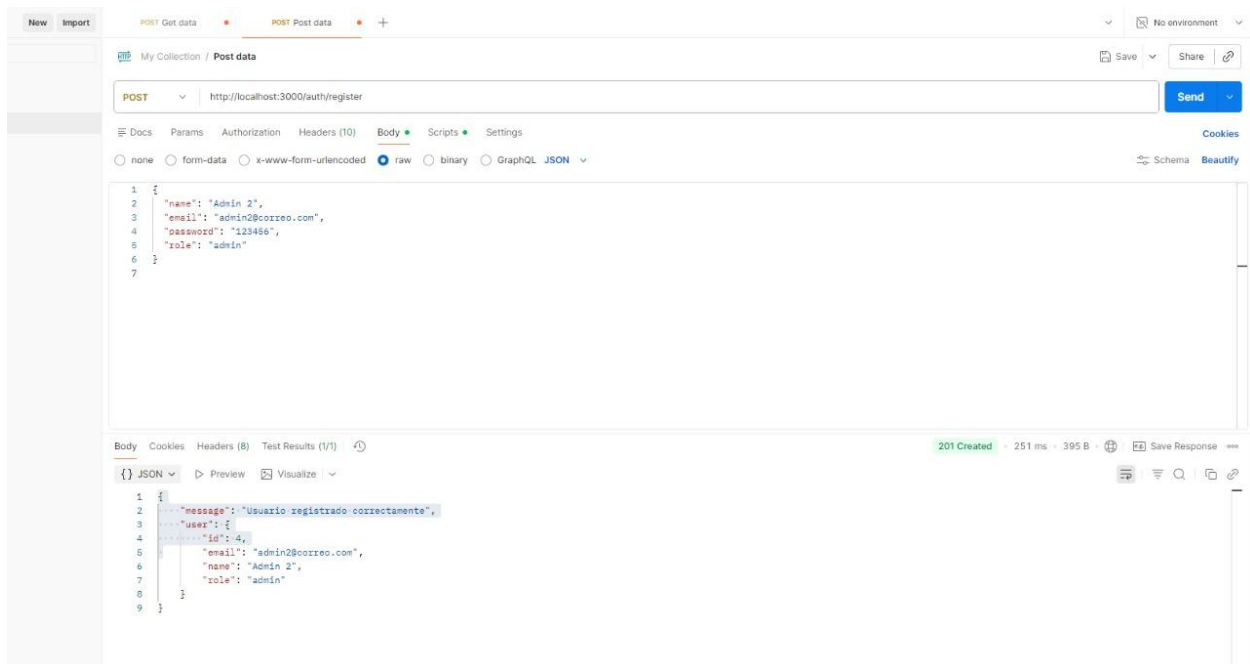
En Postman:

- **GET** <http://localhost:3000/users/me>
- El token se leyó bien
- JwtStrategy puso el user en el request
- RolesGuard dejó pasar porque la ruta acepta 'admin' y 'usuario'
- El endpoint /users/me funciona EXACTO como debe funcionar



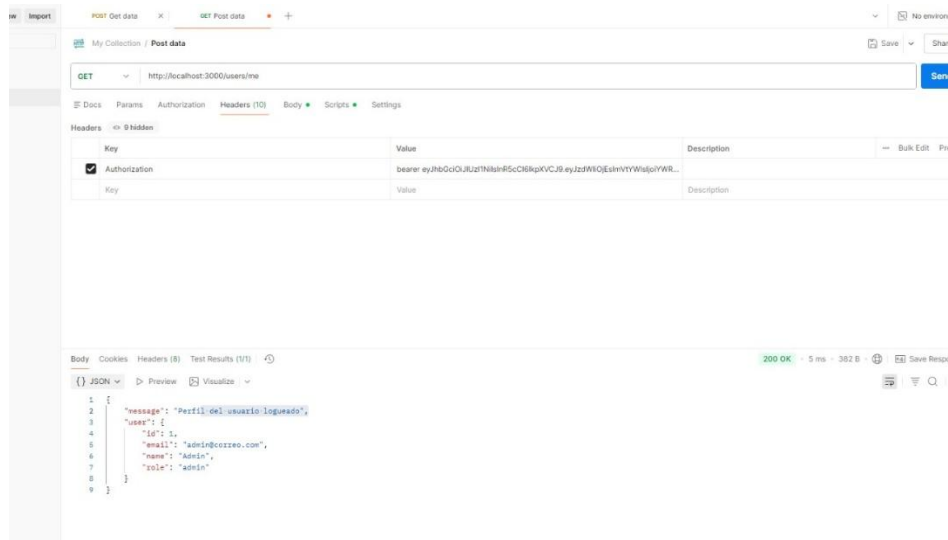
Registrar un administrador

Se registro correctamente un administrador



Prueba del login con administrador

Como vemos nos genera un token de acceso



Link del repositorio github

<https://github.com/pablo10e36366/promanage-backend>